

Project Hand-over Documentation

WExTrack System

1. Project Overview

The **WExTrack System** is a web-based application designed to manage and streamline various operational processes including ticket tracking, barangay management, customer accounts, and delivery-related configurations. The system follows an MVC architecture and is built using Java-based technologies integrated with front-end web frameworks.

This document serves as a guide for the incoming developer or team to understand the structure, functionalities, and technical aspects of the system for smooth transition and continuity.

2. System Modules

2.1 Ticketing System (Ticket Trace)

- Handles creation, tracking, and updating of tickets.
- Maintains ticket lifecycle (e.g., NCI, OPEN, RESOLVED).
- Logs interactions and timestamps for each ticket.
- Tracks assigned personnel and status changes.
-

2.2 Barangay Management

- Manages barangay records used for delivery and routing.
- Includes CRUD operations (Create, Read, Update, Delete).
- Integrated with route and location mapping.
-

2.3 Exception Waiver

- Handles exception requests and approvals.
- Allows tagging of specific tracking no. or transactions with waivers.

- Includes validation and approval workflows.

2.5 Customer Account Management

- Stores and manages customer profiles.
- Includes account creation, updates, and validation.
- Allows tagging of products, consignee and shipper address, and checklists.
-

2.6 Delivery Code Management

- Manages delivery codes used in logistics.
- Supports configuration and mapping of delivery statuses.
- Used in tracking and reporting delivery outcomes.

2.7 Dashboard and Shortcut Features

Front-end Tasks Handled

- Home Dashboard Shortcuts
- Dashboard Modules:
 - Shipment Delivery
 - Dashboard SLA
 - Dashboard on COD

Back-end Tasks Handled

- User Password Reset
- User Validation

3. Technology Stack

Front-end

- HTML
- CSS
- JavaScript

- Bootstrap

Back-end

- Java
- Spring Framework (Spring MVC)

Database

- MySQL

ORM

- Hibernate

Architecture

- MVC (Model-View-Controller)

4. System Architecture Overview

- **Model Layer:** Handles database entities and business logic (Hibernate ORM).
- **View Layer:** JSP/HTML pages styled with Bootstrap.
- **Controller Layer:** Spring MVC Controllers managing request flow and responses.

5. Key Features

- Dynamic form handling with validations
- Real-time ticket status tracking
- Modular management system (scalable)
- Role-based access (if implemented)

6. Database Overview

- Relational database using MySQL
- Key tables include:
 - ticket_trace

- barangay
- customer_account
- delivery_code
- exception_waiver

7. Setup Instructions

7.1 Prerequisites

- Java JDK (version 8)
- Apache Tomcat (or compatible server)
- MySQL Server
- IDE (Spring Tool Suite)

7.2 Steps to Run (Using SVN Repository)

1. Access SVN Repository

- Open your IDE (e.g., Spring Tool Suite).
- Navigate to SVN Repository Exploring perspective.
- Enter the repository URL and log in using your assigned SVN username and password.

2. Checkout Project

- Locate the **DashboardApp** project in the SVN repository.
- Right-click the project and select **Checkout**.
- Choose the appropriate workspace directory.

3. Import Project into IDE

- If not automatically imported, go to:
 - *File → Import → Existing Projects into Workspace*
- Select the checked-out project folder.

4. Configure Database Connection

- Update database credentials in **WEB-INF > spring > properties** > :
 - db.local.properties
- Ensure MySQL service is running.

5. Initialize Database

- Run the provided SQL scripts to create required tables and initial data.

6. Update Project Dependencies

- Right-click project → Maven → *Update Project* (if Maven-based).
- Ensure all dependencies are downloaded successfully.

7. Build and Clean Project

- Perform *Project* → *Clean* and rebuild the application.

8. Deploy to Server

- Configure Apache Tomcat in your IDE.
- Right-click project → *Run on Server* or manually add it to Tomcat.

9. Access the Application

- Open browser and navigate to:
<http://localhost:8080/DashboardApp>

10. SVN Best Practices

- Always **Update (Pull)** latest changes before starting work.
- Commit changes with proper comments.
- Resolve conflicts carefully before committing.

8. Configuration Details

- Database credentials located in:
 - db.local.properties

- Context path configurable in server settings
- Environment variables (if any) should be updated accordingly

9. Known Issues / Notes

- Ensure proper handling of null values in Hibernate mappings.
- Lazy loading may cause LazyInitializationException if not handled properly.
- Front-end responsiveness may require minor adjustments for smaller screens.
- Validation scripts are partly handled in JavaScript—review before modification.

10. Maintenance Guidelines

- Follow MVC pattern when adding new features.
- Ensure proper validation on both front-end and back-end.
- Use Hibernate best practices for database interactions.
- Maintain consistent naming conventions.
- Test all modules after any update.

11. Contact / Support

For any clarifications during the transition:

- Coordinate with previous developer or project owner.
- Review codebase comments and existing documentation.

12. Conclusion

This document provides a comprehensive overview of the **WExTrack System**. The system is modular, scalable, and designed for operational efficiency. Proper understanding of each module and adherence to the architecture will ensure smooth maintenance and future development.

Project Documentation

Project Name: WExTrack

Date: *May 06, 2026*

Created by: *John Rommel R. Rovero*